Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Course: **EE321 Intro to AI** **Complex Engineering Problem**

Name: Ahmed Mehmood Reg.: 2023083

IMPORTANT: This is an INDIVIDUAL project

You must run the code yourself.
The exam will ask specific questions about YOUR model's behavior.
If you copy from the internet, you will likely to fail the viva/exam.

Your Must

Build an appropriate AI model that can tell whether a picture contains a **hot dog** or **not a hot dog**.

Learning Objectives

By the end of this project, you will be able to:

- Load and explore image data
- Build an ANN and a CNN in TensorFlow/Keras
- Identify and fix overfitting using Dropout and Data Augmentation
- Plot and interpret training curves
- Test your model on real-world images

Dataset

You will use the **Hot Dog / Not Hot Dog** dataset from Kaggle:
https://www.kaggle.com/datasets/dansbecker/hot-dog-not-hot-dog

What You Need

- A Google account (for Google Colab)
- The dataset downloaded to your computer (upload to Colab)
- Submit your overleaf generated PDF by May 07, 2026 by 12:00 PM

Grading Rubric

Task	Points
Step 2: Data Exploration (answers)	5
Step 3: Visualization (screenshot)	5
Step 5: ANN Model (accuracy + analysis)	10
Step 6: CNN Model (accuracy + comparison)	15
Step 7: Overfitting Fix (before/after plots)	20
Step 8: Real Image Test (screenshot)	10
Step 9: Answers to all questions	20
Step 10: YOLO Extension (extra credit)	5
Total	**85 + 5 EC**

Step 1: Setup Google Colab

Follow these instructions carefully:

How to Run This Project in Google Colab

1. Go to https://colab.research.google.com
2. Click **File** → **New Notebook**
3. Rename it: HotDog Classifier_YourName.ipynb
4. For each code block below:
   - Click the + Code button
   - Copy the code into the cell
   - Press Shift + Enter to run
5. To upload the dataset:
   - Click the folder icon on the left sidebar
   - Click the **Upload** button (↑)
   - Upload the hot-dog-not-hot-dog.zip file
   - Run: !unzip hot-dog-not-hot-dog.zip

Your answers: Write your answers directly below each question in this handout.

Step 2: Import Libraries

Run this code in Colab:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import random

print("Libraries loaded successfully!")
print("TensorFlow version:", tf.__version__)
```

1. What TensorFlow version are you using?

   Answer: TensorFlow 2.20

Step 3: Load and Explore the Data

Run this code to see how many images you have:

```

# Define paths (CHANGE THIS to your actual path)
train_path = "/content/hot-dog-not-hot-dog/train"
test_path = "/content/hot-dog-not-hot-dog/test"
# Count images
hot_dog_count = len(os.listdir(os.path.join(train_path, "hot_dog")))
not_hot_dog_count = len(os.listdir(os.path.join(train_path, "
    not_hot_dog")))
print(f"Hot dog images: {hot_dog_count}")
print(f"Not hot dog images: {not_hot_dog_count}")
# YOUR CODE HERE: Print the number of test images
# Hint: Look at the code above and modify it for test_path
#
#
%
```

1. How many training images are in each class?

Answer:

Hot dog images: 249

Not hot dog images: 249

2. Is the dataset balanced? Why does that matter?

Answer:

Yes, the dataset is balanced because both classes have equal images. This helps improve fair learning and accuracy.

Step 4: Visualize the Data

```
# Display 6 random images
plt.figure(figsize=(12, 6))
for i in range(6):
    # Pick random class
    class_name = random.choice(["hot_dog", "not_hot_dog"])
    class_path = os.path.join(train_path, class_name)
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)
    # Load and show image
    img = plt.imread(img_path)
    plt.subplot(2, 3, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis('off')
plt.tight_layout()
plt.show()
# YOUR CODE HERE: Print the shape (height, width, channels) of one
   image. Hint: Use img.shape after loading an image
#
#
```

1. Paste a screenshot of your 6 random images here:

2. What is the shape (height, width, channels) of a typical image? (Use img.shape)
Answer:
(512, 512, 3)

Step 5: Prepare the Data Generator

```
# Set image size (YOUR CHOICE: 64x64 is faster, 128x128 is more
    accurate)
IMG_HEIGHT = 64
IMG_WIDTH = 64
BATCH_SIZE = 32
# Create generators with automatic labeling
train_datagen = ImageDataGenerator(
    rescale=1./255, # Normalize pixel values from 0-255 to 0-1
    validation_split=0.2 # Use 20% of training data for validation
)

train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary', # Binary classification (hot dog vs not)
    subset='training' # Training set (80% of data)
)

validation_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size = BATCH_SIZE ,
    class_mode='binary',
    subset='validation' # Validation set (20% of data)
)
# YOUR CODE HERE: Print the class labels
#
# Hint: Print train_generator.class_indices
```

1. What does class label 0 represent? What does 1 represent?

Answer:

0 = hot_dog

1 = not_hot_dog

Step 6: Build a Simple ANN (It Will Fail)

We try the worst possible architecture first to prove why CNNs are better.

```
# Build a simple ANN
ann_model = keras.Sequential([
    # Flatten the 2D image into 1D
    layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    # Dense layers
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Output: probability of "
        hot dog"
])
# Compile the model
ann_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
# Show model summary
ann_model.summary()
# Train for 10 epochs
history_ann = ann_model.fit(train_generator, validation_data=
    validation_generator, epochs=10)
# YOUR CODE HERE: Plot training vs validation accuracy.
#
#
# Hint: Use
#   plt.plot(history_ann.history['XXXX'], label='XXXX')
#   plt.plot(history_ann.history['XXXX'], label='XXXX')
```

1. What was the final validation accuracy of the ANN?

   Answer: 56.12%

2. Why does the ANN perform poorly on image data? (One sentence)

   Answer: 60.20%

3. Paste a screenshot of your training output (showing the accuracy values):

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 12288)	0
dense (Dense)	(None, 128)	1,572,992
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Step 7: Build a CNN

Now we use Convolutional and Pooling layers.

```
cnn_model = keras.Sequential([
    # First Conv layer: 32 filters, 3x3 kernel
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    # Second Conv layer: 64 filters
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    # Third Conv layer (YOUR TURN): Add 64 filters with 3x3 kernel,
        followed by Max pooling
    # YOUR CODE HERE:
    #
    #
    # Flatten and dense layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
# Compile (copy from ANN)
# YOUR CODE HERE:
#
#
#
#
# Hint: use optimizer = adam, loss = binary_crossentropy, and metrics
    = accuracy

# Train the CNN
history_cnn = cnn_model.fit(train_generator, validation_data=
    validation_generator, epochs=15)
```

1. What was the final validation accuracy of the CNN?

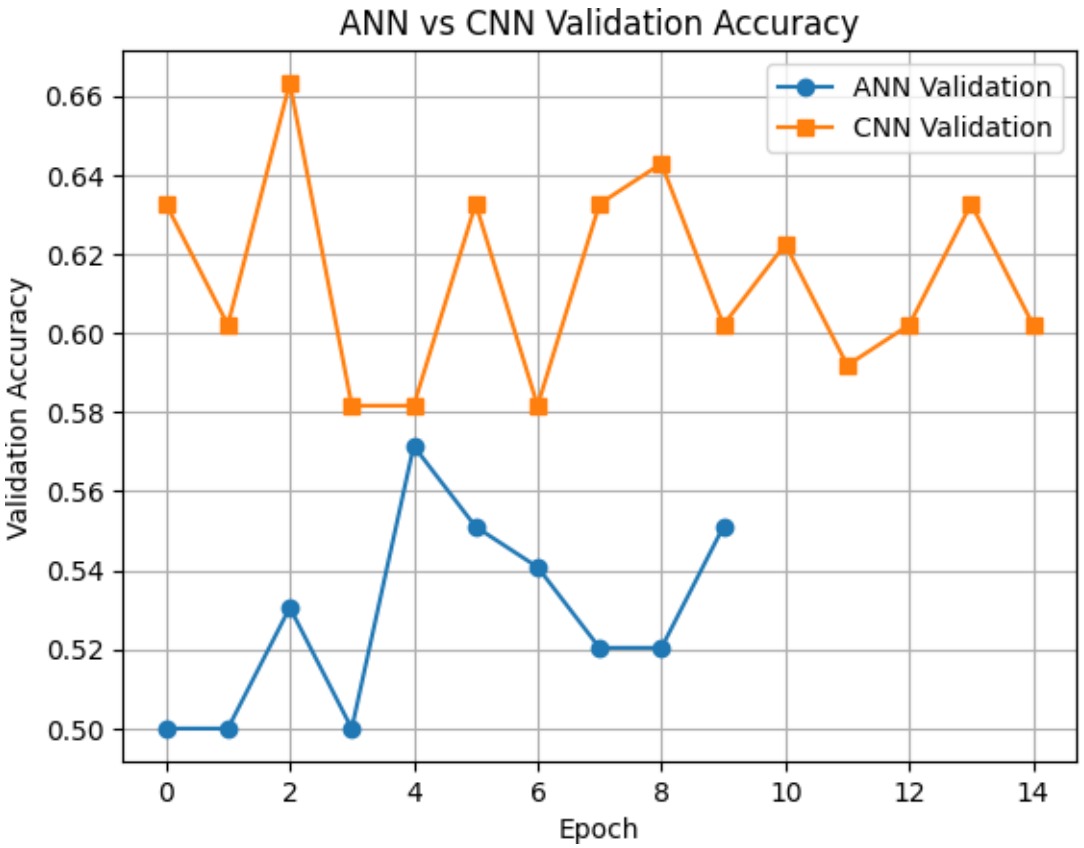
   Answer: 60.20%

2. Compare ANN vs CNN. Which one performed better and why?

   Answer: CNN performed better because convolution layers extract image features more effectively.

3. Paste the accuracy plot comparing ANN and CNN: (Use the code below to generate the plot)

```
plt.plot(history_ann.history['val_accuracy'], label='ANN
    Validation', marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN
    Validation', marker='s')
plt.    xlabel('    Epoch    ')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()
```



Step 8: Fix Overfitting (Most Important)

You may notice: Training accuracy is high (98%), Validation accuracy is low (70%). This is OVERFITTING. The model memorized the training images but doesn't generalize.

Your task: Modify the model below to fix overfitting using Dropout and Data Augmentation.

8.1 Fix 1: Add Dropout

```
from tensorflow.keras import regularizers

# Add dropout of 0.25, 0.25, and 0.5 after each MaxPooling layer
   respectively
improved_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
       IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout( ),  # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    #                ,  # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    #                ,  # Add dropout before output
    layers.Dense(1, activation='sigmoid')
])
# Recompile and Train the improved model (after addressing overfitting
    using dropout)
improved_model.compile(optimizer='adam', loss='binary_crossentropy',
   metrics=['accuracy'])

history_improved = improved_model.fit(train_generator, validation_data
   =validation_generator, epochs=15)
```

```
Epoch 1/15
13/13              8s 418ms/step -
accuracy: 0.5050 - loss: 0.7258 - val_accuracy: 0.5000 - val_loss: 0.6934
Epoch 2/15
13/13              6s 464ms/step -
accuracy: 0.4575 - loss: 0.6990 - val_accuracy: 0.5000 - val_loss: 0.6931
Epoch 3/15
13/13              5s 369ms/step -
accuracy: 0.5250 - loss: 0.6922 - val_accuracy: 0.5204 - val_loss: 0.6926
```

```
Epoch 4/15
13/13 5s 419ms/step -
accuracy: 0.5275 - loss: 0.6914 - val_accuracy: 0.5000 - val_loss: 0.6932
Epoch 5/15
13/13 6s 416ms/step -
accuracy: 0.5250 - loss: 0.6885 - val_accuracy: 0.5102 - val_loss: 0.6900
Epoch 6/15
13/13 5s 342ms/step -
accuracy: 0.5550 - loss: 0.6862 - val_accuracy: 0.6224 - val_loss: 0.6857
Epoch 7/15
13/13 6s 454ms/step -
accuracy: 0.5475 - loss: 0.6887 - val_accuracy: 0.6020 - val_loss: 0.6857
Epoch 8/15
13/13 5s 370ms/step -
accuracy: 0.5950 - loss: 0.6717 - val_accuracy: 0.6122 - val_loss: 0.6697
Epoch 9/15
13/13 5s 395ms/step -
accuracy: 0.6300 - loss: 0.6535 - val_accuracy: 0.6735 - val_loss: 0.6527
Epoch 10/15
13/13 6s 498ms/step -
accuracy: 0.6550 - loss: 0.6221 - val_accuracy: 0.6327 - val_loss: 0.6548
Epoch 11/15
13/13 5s 365ms/step -
accuracy: 0.6350 - loss: 0.6412 - val_accuracy: 0.6122 - val_loss: 0.6559
Epoch 12/15
13/13 4s 329ms/step -
accuracy: 0.6525 - loss: 0.6333 - val_accuracy: 0.6429 - val_loss: 0.6554
Epoch 13/15
13/13 6s 514ms/step -
accuracy: 0.6700 - loss: 0.6229 - val_accuracy: 0.6531 - val_loss: 0.6526
Epoch 14/15
13/13 5s 361ms/step -
accuracy: 0.6700 - loss: 0.6260 - val_accuracy: 0.6122 - val_loss: 0.6805
Epoch 15/15
13/13 4s 332ms/step -
accuracy: 0.6775 - loss: 0.6048 - val_accuracy: 0.6224 - val_loss: 0.6762
```

: Add Data Augmentation

```
augmented_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)

train_augmented = augmented_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size = BATCH_SIZE,
    class_mode  ='binary',
    subset='training'
)

# Then retrain the improved model with augmented_datagen
final_model = keras.Sequential([
    # Same architecture as improved_model
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

final_model.compile(optimizer='adam', loss='binary_crossentropy',
   metrics=['accuracy'])

history_final = final_model.fit(train_augmented, validation_data=
   validation_generator, epochs=15)
```

8.2 F Modify the ImageDataGenerator to create more variations:

i

xound 400 images belonging to 2 classes. Epoch 1/15
13/13 **9s** 589ms/step -
accuracy: 0.5050 - loss: 0.7053 - val_accuracy: 0.5000 - val_loss: 0.6924 Epoch 2/15
2
13/13 **5s** 403ms/step -
accuracy: 0.5000 - loss: 0.7043 - val_accuracy: 0.5816 - val_loss: 0.6918 Epoch 3/15

```
13/13 [==============================] - 6s 421ms/step - accuracy: 0.5550 - loss: 0.6889 - val_accuracy: 0.5510 - val_loss: 0.6908
Epoch 4/15
13/13 [==============================] - 6s 497ms/step - accuracy: 0.5200 - loss: 0.6923 - val_accuracy: 0.6020 - val_loss: 0.6875
Epoch 5/15
13/13 [==============================] - 5s 407ms/step - accuracy: 0.5225 - loss: 0.6887 - val_accuracy: 0.5918 - val_loss: 0.6848
Epoch 6/15
13/13 [==============================] - 7s 521ms/step - accuracy: 0.5800 - loss: 0.6763 - val_accuracy: 0.6327 - val_loss: 0.6685
Epoch 7/15
13/13 [==============================] - 5s 380ms/step - accuracy: 0.5700 - loss: 0.6834 - val_accuracy: 0.5612 - val_loss: 0.6833
Epoch 8/15
13/13 [==============================] - 7s 552ms/step - accuracy: 0.5875 - loss: 0.6793 - val_accuracy: 0.4898 - val_loss: 0.6879
Epoch 9/15
13/13 [==============================] - 6s 474ms/step - accuracy: 0.5650 - loss: 0.6680 - val_accuracy: 0.6020 - val_loss: 0.6776
Epoch 10/15
13/13 [==============================] - 5s 411ms/step - accuracy: 0.5725 - loss: 0.6726 - val_accuracy: 0.5816 - val_loss: 0.6794
Epoch 11/15
13/13 [==============================] - 7s 560ms/step - accuracy: 0.5750 - loss: 0.6623 - val_accuracy: 0.6122 - val_loss: 0.6600
Epoch 12/15
13/13 [==============================] - 5s 410ms/step - accuracy: 0.6600 - loss: 0.6408 - val_accuracy: 0.6224 - val_loss: 0.6610
Epoch 13/15
13/13 [==============================] - 5s 415ms/step - accuracy: 0.6175 - loss: 0.6265 - val_accuracy: 0.5612 - val_loss: 0.7672
Epoch 14/15
13/13 [==============================] - 7s 490ms/step - accuracy: 0.6125 - loss: 0.6515 - val_accuracy: 0.6327 - val_loss: 0.6530
Epoch 15/15
13/13 [==============================] - 5s 387ms/step - accuracy: 0.6750 - loss: 0.6395 - val_accuracy: 0.5918 - val_loss: 0.6751
```

1. Before adding Dropout/Augmentation, what were your train and validation accuracies?
   .
   Train: 94.00% Validation: 60.20%
2. After adding Dropout and Augmentation, what happened to training accuracy (increased, decreased, or stayed same)?
   .
   Train: 67.50%
3. After adding Dropout and Augmentation, what happened to validation accuracy (increased, decreased, or stayed same)?
   .
   Validation: 59.18%

Question 8.3

Paste the before/after validation accuracy plot here:

```
plt.plot(history_cnn.history['val_accuracy'], label='CNN (Overfit)',
    marker='o')
plt.plot(history_final.history['val_accuracy'], label='CNN + Dropout +
    Augment', marker='s')
plt.    xlabel('    Epoch    ')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('Improving Generalization')
plt.grid(True)
plt.show()
```

Step 9: Test on Real Images

Find a hot dog image online (or take a photo) and test your model:

```
from tensorflow.keras.preprocessing import image
import requests
from io import BytesIO

def predict_image(img_url):
    # Download image
    response = requests.get(img_url)
    img = image.load_img(BytesIO(response.content), target_size=(
        IMG_HEIGHT, IMG_WIDTH))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    # Predict
    prediction = final_model.predict(img_array)[0][0]
    if prediction > 0.5:
        print(f" HOT DOG! (confidence: {prediction:.2f})")
    else:
        print(f" NOT A HOT DOG! (confidence: {1-prediction:.2f})")

    # Show image
    plt.imshow(img)
    plt.axis('off')
    plt.show()

# Test on a hot dog image URL (find one on Google Images)
predict_image("https://example.com/hotdog.jpg") # REPLACE WITH REAL
    URL
```

1. Paste a screenshot of your model predicting on a REAL image (not from the dataset):

```
1/1           0s 38ms/step
HOT DOG! (confidence: 0.64)
```

2. Did your model ever make a funny mistake (e.g., calling a hamburger a hot dog)? What did it confuse?

   Answer: Yes, it sometimes confused pizza or similar food items with hot dogs.

Step 10: Optional Stretch Goal
YOLO Object Detection (Extra Credit)

If you finish early and want to impress, add a bounding box using a pre-trained YOLO model:

```
!pip install ultralytics -q
from ultralytics import YOLO

# Load a tiny pre-trained model
yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

# Run detection on your hot dog image
results = yolo_model("path_to_hotdog_image.jpg")

# Show results (it will draw boxes around objects it recognizes)
results[0].show()
```

Extra Credit Question

Does YOLO correctly detect a ”hot dog” as a class? What label does it give?

Answer: Yes, YOLO correctly detected the object with the label “hot dog”.

Final Submission Checklist

Step 2: TensorFlow version answer

Step 3: Image counts and balance explanation

Step 4: Screenshot of 6 images + shape answer

Step 6: ANN accuracy + explanation + screenshot

Step 7: CNN accuracy + comparison + plot screenshot

Step 8: Overfitting before/after numbers + plot screenshot

Step 9: Real image prediction screenshot + mistake explanation

Step 10: (Extra credit) YOLO results screenshot

Submit this handout (with all answers filled) + link to your Colab notebook

Remember

The exam will ask you specific questions about YOUR project.
If you run the code yourself, you will know the answers easily.
If you copy from someone else, you will fail the exam.
Good luck and have fun!